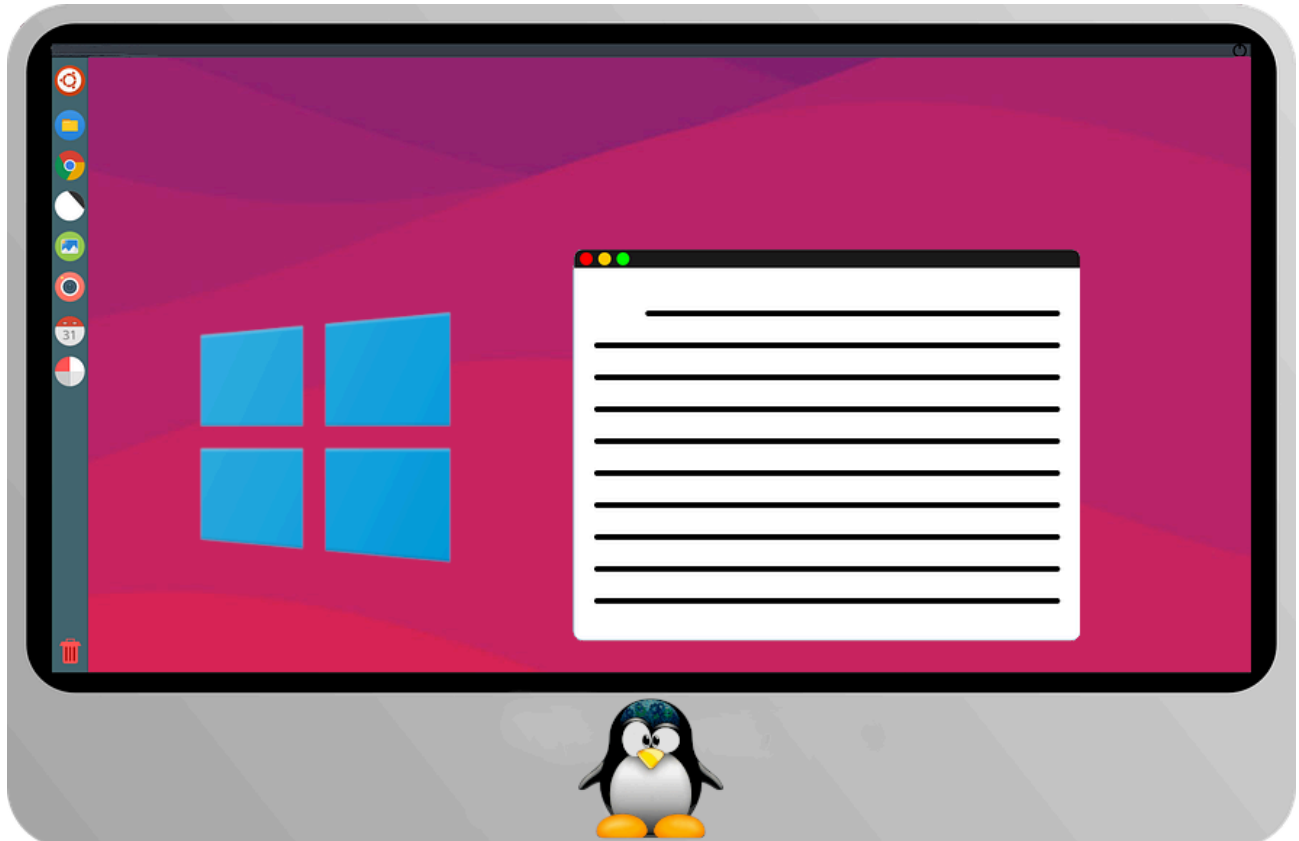


Domain Join Ubuntu 22.04 to Active Directory – Michael Waterman

 michaelwaterman.nl/2022/10/02/domain-join-ubuntu-22-04-to-active-directory

Michael Waterman

October 2, 2022



Sometimes you'll be needing a setup that includes both Windows and Linux based machines for managing the infrastructure and data processing. Although joining a Windows machine to an Active Directory domain is fairly simple, joining a Linux based systems requires a little more effort. This posts focusses on joining Ubuntu based nodes (Desktop or Server systems) to a Windows Active Directory domain for simplified management and a unified login experience.

Turns out that there are a couple of way of accomplishing this task. It's not really that straightforward as it is with Windows but once you get the right tools and know what files to edit it's really not that hard. With this post I want to share my personal experiences and show you step-by-step on how to add a Linux based host to a Windows Active Directory.

System Security Services Daemon

I've tried a couple of options/packages for joining a Linux machine into a Windows based Active Directory domain, but in the end, for me, using the System Security Services Daemon (SSSD) was the most effective way to accomplish my task at hand. The SSSD is like the intermediary that helps you to configure the system without you needing to know

what files you need to edit (Although it can be very useful). The other benefit that I discovered is that it's available on all major distributions, like RedHat or Ubuntu. So What I will be describing here will be useful in many situations. Let's dive in.

Lab infrastructure

My lab setup is straightforward. Two Domain Controllers, named “*LAB-DC01* (192.168.66.2)” & “*LAB-DC02* (192.168.66.3)” in the “*water.lab*” domain. Next to the ADDC role both machines also hosts the DNS role. The client computer is an Ubuntu 22.04 machine, named “*Ubuntu*”, and is configured to use the DNS server on “*LAB-DC01*”. I've checked connectivity to “*LAB-DC01*” with a simple ICMP ping and name resolution with NSLookup. Both work as expected.

Hostname

First thing that we need to check before joining the machine is the hostname. Although Ubuntu can handle long hostnames, a Windows machine is restricted to 15 characters max because of the legacy protocol NetBios. In turn that means that our Ubuntu client machines is also restricted to the same length for it's hostname. Check the current hostname with this command:

In my case this resulted in the following output:

```
superuser@ubuntu:~$ hostnamectl
Static hostname: ubuntu.water.lab
Icon name: computer-vm
Chassis: vm
Machine ID: 45b9c881311e4b41a584539d5bd31156
Boot ID: c79e064bbb3c47b781e49b0e7704a727
Virtualization: kvm
Operating System: Ubuntu 22.04.1 LTS
Kernel: Linux 5.15.0-48-generic
Architecture: x86-64
Hardware Vendor: QEMU
Hardware Model: Standard PC _Q35 + ICH9, 2009_
```

In case you need to change the name, use the following command and reboot afterwards:

```
sudo hostnamectl set-hostname <hostname.domain.suffix>
```

Hint! This command will not change the hostname in the “*/etc/hosts*” file.

Tip! Changing the hostname can also be done by editing these files, reboot afterwards.

```
sudo nano /etc/hostname
sudo nano /etc/hosts
```

Networking

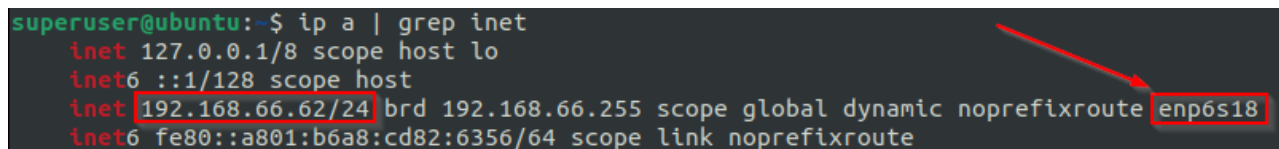
Although it is preferred to have DHCP handle the assignment of the IP address, Gateway and DNS servers, it can sometimes be necessary to assign these settings manually, for example when you need a static IP address or there's no DHCP server available. Ubuntu uses “*Netplan*” to handle network configurations that are stored in a YAML file in the directory “*/etc/netplan*”. Open the present YAML file and edit the file based on your requirements or take the following configuration as an example:

```
sudo nano /etc/netplan/01-network-manager-all.yaml
```

01-network-manager-all.yaml content:

```
# Let NetworkManager manage all devices on this system
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    enp6s18:
      dhcp4: no
      dhcp6: no
      addresses: [192.168.66.98/24]
      routes:
        - to: default
          via: 192.168.66.1
          metric: 100
      nameservers:
        addresses: [192.168.66.2, 192.168.66.3]
        search: [water.lab]
```

Pay special attention to the indentation of the file as YAML is very picky to how the file is formatted. There are a few things that you probably want to change. First thing is the property “*enp6s18*”. This represents the name of my network adapter, yours could be named differently. There are multiple ways to get the name, I use the following command:



```
superuser@ubuntu:~$ ip a | grep inet
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 192.168.66.62/24 brd 192.168.66.255 scope global dynamic noprefixroute enp6s18
inet6 fe80::a801:b6a8:cd82:6356/64 scope link noprefixroute
```

In the output, the latter part of the line with your IP address has the name of your network adapter.

Another property is the default gateway. In the version of “*Netplan*” that ships with Ubuntu 22.04 the default route is configured in the “*routes*” property. Editing the value after “*via*” changes the default route. Use the following command to get the current configured default route:

```
superuser@ubuntu:~$ ip r | grep default
default via 192.168.66.1 dev enp6s18 proto dhcp metric 100
superuser@ubuntu:~$
```

Last but certainly not least let's focus on name resolution, also known as DNS. Without it we can't do a whole lot, certainly not join a domain. The property that needs to be changed is "*nameservers – addresses*". To get the current values, use the command:

```
superuser@ubuntu:~$ resolvectl status
Global
  Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  resolv.conf mode: stub

Link 2 (enp6s18)
  Current Scopes: DNS
  Protocols: +DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  Current DNS Server: 192.168.66.2
  DNS Servers: 192.168.66.2 192.168.66.3
  DNS Domain: water.lab
```

Once you're happy with the netplan configuration, you need to apply it. This can simply be done by executing the command:

if no errors are visible on the screen the configuration is successfully applied. Use the previous commands to verify the configuration.

Timing is everything

Using Kerberos authentication relies heavily on the correct time being set at both ends. It should always be within a maximum of 5 minutes difference between the two entities trying to authenticate. On Ubuntu, "*timesyncd*" is responsible for all things related with time. First we need to point the client to the closest time source. Usually this is the DC that will provide the correct time, but any time source will do, as long as everything is in sync. Edit the following file to add the NTP source as displayed in the example:

```
sudo nano /etc/systemd/timesyncd.conf
```

Hint! setting the value for "*RootDistanceMaxSec*" prevents the "*Server has too large root distance. Disconnecting*" error message and the time service not syncing.

Set the NTP sync to true

```
sudo timedatectl set-ntp on
```

Bash

Restart the service

```
sudo systemctl restart systemd-timesyncd.service
```

Bash

Force Sync

```
sudo timedatectl --adjust-system-clock
```

Bash

Check status

```
timedatectl status
```

Bash

Required packages

First thing we need to do is install all the appropriate packages. This post is focus on Ubuntu 22.04, but it's almost the same on other distributions that use apt (or yum) as their package manager. Open up a terminal, gain root privileges, install these packages:

- realmd
- sssd
- sssd-tools
- libnss-sss
- libpam-sss
- krb5-user
- adcli
- samba-common-bin

```
sudo apt -y install realmd sssd sssd-tools libnss-sss libpam-sss krb5-user adcli  
samba-common-bin
```

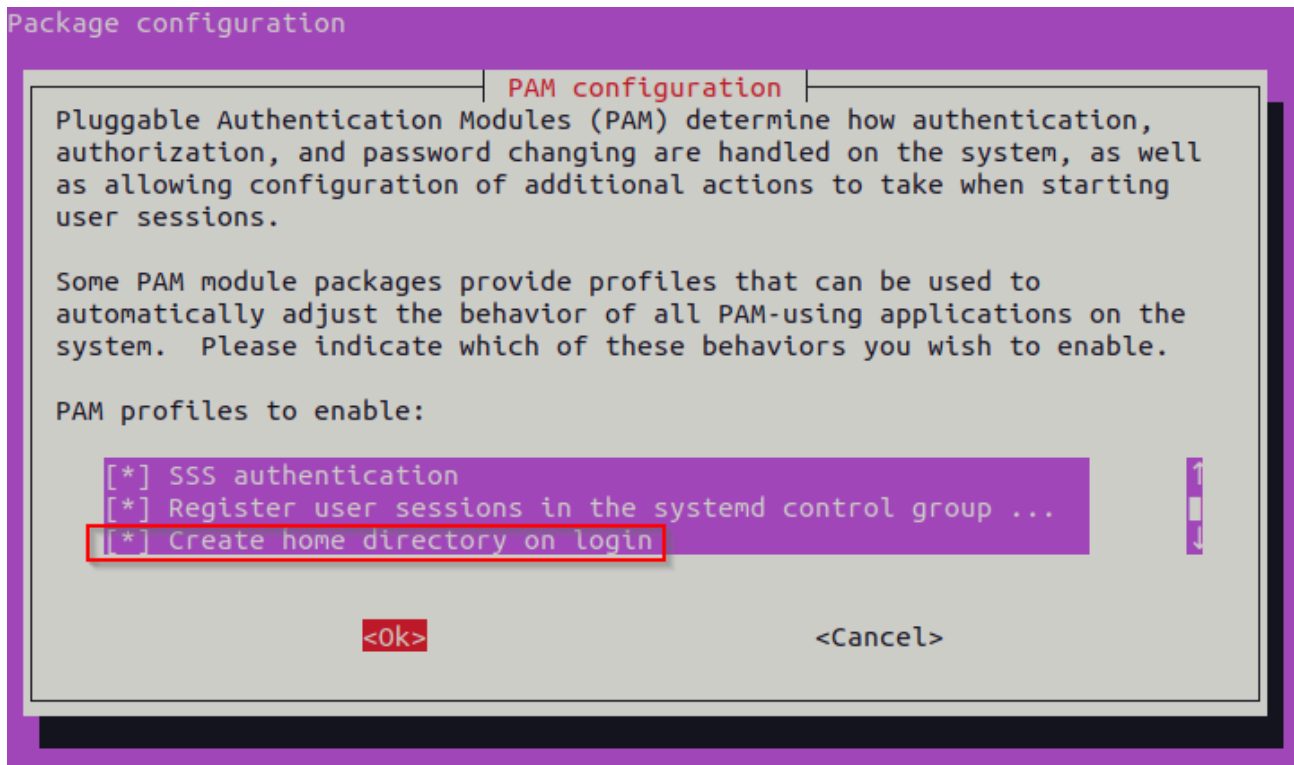
Bash

Auto create the home directory

On Ubuntu every user that does an interactive logon to the system needs a home directory. For domain users this needs to be set before a user is able to successfully logon and start working. You can accomplish this tasks in two ways, I'll show them both.

First up is a guided way of doing things. In a terminal type:

this will bring up a dialog screen for editing the configuration file `"/etc/pam.d/common-session"`.



Scroll down up to the point where it states :*"Create home directory on login"*. Use the space bar to select, tab to "OK" and hit enter. If you want to do it in a more scriptable way use the following command-line option:

```
sudo pam-auth-update --enable mkhomedir
```

Bash

Hint! The information is stored in the file: *"/etc/pam.d/common-session"*.

```
session optional pam_mkhomedir.so
```

Bash

Testing Active Directory access

Now that the all the packages have been installed and configured with the appropriate settings, we're ready to test the setup. Ubuntu has a few very useful tools to see if Kerberos authentication will succeed. Use the following set of commands to test authentication:

Discover the domain

```
realm discover water.lab
```

Bash

Get a Kerberos ticket for the user Administrator

```
kinit administrator
```

Bash

Display the Kerberos ticket

klist

Bash

```
superuser@ubuntu:~$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: administrator@WATER.LAB

Valid starting    Expires          Service principal
02-10-22 20:03:50 03-10-22 06:03:50 krbtgt/WATER.LAB@WATER.LAB
        renew until 03-10-22 20:03:45
superuser@ubuntu:~$
```

Destroy the Kerberos ticket

kdestroy

Bash

The reason I destroy the ticket first is that it will otherwise be used during the domain join that I'll show you next. The domain account used for joining the domain isn't necessarily the same account as testing the authentication. Besides it's also a good way to learn all the available tools.

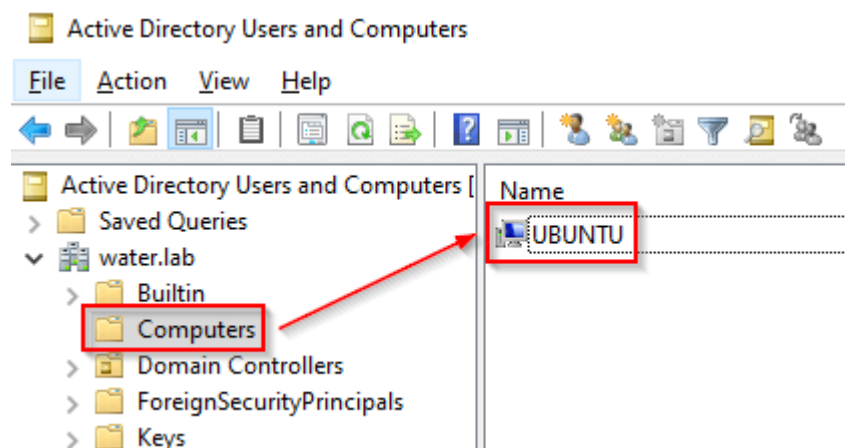
Joining the Active Directory domain

And now the part we've all been waiting for, joining the domain. Actually this is the easy part, if everything is configured as it should, the domain join should be fairly easy to accomplish. Simply run the following command in a terminal:

```
sudo realm join --verbose --user=administrator water.lab
```

Bash

If there are no errors displayed at the end, the machine is successfully joined to the domain! On a Windows PC open “*Active Directory Users and Computers*” and check if a computer object is created.



Logon with Active Directory credentials

before we login, wouldn't it be cool to only use the name of the user instead of the entire "User Principal Name" (UPN)? So instead of "michael@water.lab", only "michael" as the username? There is a neat little trick for that. Run the following command to edit the "System Security Services Daemon" configuration file:

```
sudo nano /etc/sss/sss.conf
```

Bash

Hint! The "sss.conf" file is automatically created after the domain join. It will not exist before joining the machine to the domain.

set the line "use_fully_qualified_names" to "False".

```
[sss]
domains = water.lab
config_file_version = 2
services = nss, pam

[domain/water.lab]
default_shell = /bin/bash
krb5_store_password_if_offline = True
cache_credentials = True
krb5_realm = WATER.LAB
realmd_tags = manages-system joined-with-adcli
id_provider = ad
fallback_homedir = /home/%u@d
ad_domain = water.lab
use_fully_qualified_names = False
ldap_id_mapping = True
access_provider = ad
```

Hint! You need to reboot or restart the sssd daemon before this setting is in effect.

```
sudo systemctl restart sssd
```

Bash

As a final step, on the logon screen, click the option: "Not listed", and fill in your credentials. If everything is configured according to plan, you should be able to logon and a home directory is automatically created for the user.

To verify if everything has gone successful, open up a terminal and issue the "klist" command. This will display your ticket granting ticket (TGT), also known as the key to your kingdom.


```
michael@ubuntu:~$ klist
Ticket cache: FILE:/tmp/krb5cc_1218803102_gV8KxP
Default principal: michael@WATER.LAB

Valid starting    Expires          Service principal
02-10-22 20:59:30  03-10-22 06:59:30  krbtgt/WATER.LAB@WATER.LAB
        renew until 03-10-22 20:59:30
michael@ubuntu:~$
```

That's it for this post. Next up will be some more advanced permissions and configuration.